UNIVERSITY INSTITUTE OF COMPUTING

MINI PROJECT REPORT

Subject: Python Programming

SUBMITED BY:

Name: Bhawna Bisht

UID: 24BSC10007

Course: BSC(CS)

Section: 24BSC-1A

SUBMITTED TO:

Name: Mrs. Monika

Designation: Asst. Professor

1. Introduction

The Student Record Management System is a simple Python-based application developed to manage student information efficiently. Educational institutions need to maintain student details such as roll number, name, age, course, and marks. Managing these records manually can be time-consuming and make searching, updating, and deleting information difficult.

This project provides a menu-driven system that allows users to add new student records, view all records, search for a particular student, update existing information, and delete records. The application uses Python dictionaries to store student details and functions to perform different operations.

The project demonstrates fundamental Python programming concepts, including dictionaries, functions, loops, conditional statements, and user input. It is designed as a beginner-level project to understand how programming can simplify basic record management.

2. Features

The Student Record Management System provides the following features:

- Add Student Record: Allows users to enter a student's roll number, name, age, course, and marks.
- View All Records: Displays all student records currently stored in the system.
- Search Student: Finds a student's information using their roll number.
- Update Student Record: Allows users to modify the details of an existing student.
- Delete Student Record: Removes a student's information using their roll number.
- Duplicate Record Checking: Prevents adding a student with a roll number that already exists.
- Menu-Driven Interface: Displays a numbered menu for easy navigation.
- Exit Option: Allows users to terminate the program when they finish.

The program uses built-in Python functionality and does not require an external database or additional libraries.

3. Workflow and Algorithm

3.1 Workflow

The application follows a simple sequence of operations to manage student records.

1. Program Initialization: An empty dictionary named students is created.
2. Display Menu: The program displays options for adding, viewing, searching, updating, deleting, and exiting.
3. User Input: The user selects an operation by entering a menu number.
4. Process Selection: The program calls the corresponding function.
5. Record Management: Student information is added, displayed, searched, updated, or deleted as required.
6. Display Result: A success or error message is displayed based on the operation.
7. Repeat or Exit: The menu appears again until the user selects the exit option.

3.2 Algorithm

Algorithm: Student Record Management System

1. Start the program.
2. Create an empty dictionary to store student records.
3. Display the main menu.
4. Accept the user's choice.
5. If the choice is 1, add a new student record.
6. If the choice is 2, display all student records.
7. If the choice is 3, search for a student using the roll number.
8. If the choice is 4, update an existing student record.
9. If the choice is 5, delete a student record.
10. If the choice is 6, terminate the program.
11. If the choice is invalid, display an error message.
12. Repeat the menu until the user exits.

4. Source Code

The following Python code implements the complete Student Record Management System.

```
students = {}
def add_student():
    roll_no = input("Enter Roll Number: ")
    if roll_no in students:
        print("Student already exists!")
        return

    name = input("Enter Student Name: ")
    age = int(input("Enter Age: "))
    course = input("Enter Course: ")
    marks = float(input("Enter Marks: "))

    students[roll_no] = {
        "name": name,
        "age": age,
        "course": course,
        "marks": marks

    print("Student added successfully!")

def view_students():
    if not students:
        print("No student records found.")
        return
```

```
        for roll_no, details in students.items():
        print("\nRoll Number:", roll_no)
        print("Name:", details["name"])
        print("Age:", details["age"])
        print("Course:", details["course"])
        print("Marks:", details["marks"])

def search_student():
    roll_no = input("Enter Roll Number to search: ")

    if roll_no in students:
        print("Student Details:", students[roll_no])
    else:
        print("Student not found!")

def update_student():
    roll_no = input("Enter Roll Number to update: ")

    if roll_no in students:
        name = input("Enter New Name: ")
        age = int(input("Enter New Age: "))
        course = input("Enter New Course: ")
        marks = float(input("Enter New Marks: "))

        students[roll_no] = {
            "name": name,
            "age": age,
```

```
            "course": course,
            "marks": marks
        }

        print("Student record updated!")
    else:
        print("Student not found!")

def delete_student():
    roll_no = input("Enter Roll Number to delete: ")

    if roll_no in students:
        del students[roll_no]
        print("Student record deleted!")
    else:
        print("Student not found!")

while True:
    print("\n===== STUDENT RECORD SYSTEM =====")
    print("1. Add Student")
    print("2. View All Students")
    print("3. Search Student")
    print("4. Update Student")
    print("5. Delete Student")
    print("6. Exit")

    choice = input("Enter your choice: ")
```

```
    if choice == "1":
        add_student()
    elif choice == "2":
        view_students()
    elif choice == "3":
        search_student()
    elif choice == "4":
        update_student()
    elif choice == "5":
        delete_student()
    elif choice == "6":
        print("Thank you for using Student Record System!")
        break
    else:
        print("Invalid choice! Please try again.")
```

5. Key Code Implementation

5.1 Dictionary-Based Data Storage

The program uses a Python dictionary named students to store student records. Each roll number acts as a unique key, while the corresponding value contains the student's details.

```
students = {}

students[roll_no] = {
    "name": name,
    "age": age,
    "course": course,
    "marks": marks
}
```

This structure makes it convenient to access a student's information using their roll number.

5.2 Duplicate Record Checking

Before adding a student, the program checks whether the roll number already exists in the dictionary.

```
if roll_no in students:
    print("Student already exists!")
    return
```

This prevents duplicate roll numbers from being added through the add_student() function.

5.3 Searching for a Student

The search operation checks whether the entered roll number exists. If found, the program displays the corresponding student details.

```
if roll_no in students:
    print("Student Details:", students[roll_no])
else:
    print("Student not found!")
```

\

5.4 Updating Student Information

The update function collects new information and replaces the existing record associated with the selected roll number.

```
students[roll_no] = {
    "name": name,
    "age": age,
    "course": course,
    "marks": marks
}
```

5.5 Deleting a Student Record

The del statement removes an existing student record from the dictionary.

```
if roll_no in students:
    del students[roll_no]
    print("Student record deleted!")
```

if roll_no in students: del students[roll_no] print("Student record deleted!")

5.6 Menu-Driven Execution

The while True loop repeatedly displays the menu. The break statement terminates the loop when the user selects Exit.

```
while True:
    # Display menu and accept user choice

    if choice == "6":
        break
```

This allows users to perform multiple operations during one program session.

6. Output and Experimental Testing

6.1 Sample Output

The following example demonstrates adding a student record.

```
===== STUDENT RECORD SYSTEM =====
1. Add Student
2. View All Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit

Enter your choice: 1
Enter Roll Number: 101
Enter Student Name: Rahul
Enter Age: 20
Enter Course: BCA
Enter Marks: 85

Student added successfully!
```

6.2 Viewing Student Records

After adding a student, selecting option 2 displays the stored record.

```
Enter your choice: 2

Roll Number: 101
Name: Rahul
Age: 20
Course: BCA
Marks: 85.0
```

6.3 Experimental Testing

The program can be tested using the following test cases.

Test Case	Operation	Expected Result
TC01	Add a new student	Record is added successfully
TC02	Add a duplicate roll number	Duplicate record is rejected
TC03	View all records	Stored student details are displayed
TC04	Search for an existing roll number	Matching record is displayed
TC05	Search for a nonexistent roll number	Student not found message appears
TC06	Update an existing record	Student details are updated
TC07	Delete an existing record	Student record is deleted
TC08	Enter an invalid menu choice	Invalid choice message appears
TC09	Select Exit	Program terminates

These test cases cover the main functions of the system. Actual results should be confirmed by running the program and performing each operation.

Testing limitation: The current version does not validate all incorrect inputs. For example, entering text instead of a number for age may cause an error. Also, records are stored only in memory and disappear when the program exits.

7. Future Aspects and Improvements

The Student Record Management System can be improved in several ways to make it more reliable and useful.

- Permanent Data Storage: File handling or an SQLite database can be used to save records permanently.
- Input Validation: Validation can be added to ensure that age and marks contain valid values.
- Graphical User Interface: Tkinter can be used to create a graphical interface with buttons, text fields, and tables.
- Grade Calculation: The program can calculate grades based on students' marks.
- Report Generation: Student report cards can be generated with marks, grades, and other details.
- Login System: Authentication can be added to restrict access to authorised users.
- Sorting and Filtering: Records can be sorted by name, marks, or course.
- Data Export: Student records can be exported to CSV or Excel files for further analysis.

These improvements would help transform the basic program into a more complete student information management application.

8. Conclusion

The Student Record Management System demonstrates how Python programming can be applied to manage student information through a simple menu-driven application. It provides essential operations such as adding, viewing, searching, updating, and deleting records.

The project uses dictionaries for data storage, functions for modular programming, conditional statements for decision-making, and loops for repeated execution. It also demonstrates the basic CRUD operations commonly used in information management systems.

Although the current version is suitable for learning and small demonstrations, it has limitations because the records are stored temporarily in memory and input validation is limited. Adding permanent storage, a graphical interface, and stronger validation would improve its functionality.

Overall, this project provides practical experience in Python programming and establishes a foundation for developing more advanced record management systems.

9. References

1. Python Software Foundation. Python 3 Documentation. https://docs.python.org/3/
2. Python Software Foundation. The Python Tutorial. https://docs.python.org/3/tutorial/
3. Python Software Foundation. Built-in Types — Dictionaries. https://docs.python.org/3/library/stdtypes.html#mapping-types-dict
4. Python Software Foundation. Control Flow Tools. https://docs.python.org/3/tutorial/controlflow.html
5. Course notes and classroom learning materials on Python programming.